

Tablice dynamiczne, zarządzane pamięcią

1. Zrobić wersję zadania nr 3 z poprzedniego arkusza, ale z użyciem tablicy dynamicznej:

1. deklarujemy na początku pustą tablicę (`Produkt* storage_array`), zapisujemy dotychczasową długość: `int n_items=0` .
2. Wczytujemy kolejne linijki z `magazyn.txt` . Dopóki nie wczytamy całości:
3. Jeśli `n_items==0` , zwiększamy pojemność tablicy (za pomocą `malloc`) o `sizeof(Produkt)` bajtów (`storage_array = malloc(sizeof(Produkt))`).
4. W przeciwnym wypadku (`n_items>0`), zwiększamy pojemność tablicy (za pomocą `realloc`), tak aby pomieścić `n_items+1` produktów: `storage_array= realloc(storage_array, (n_items+1)*sizeof(Produkt))` .
5. Zapisujemy dane nowego produktu (z pobraniem linijki) na koniec tablicy (indeks `n_items`).
6. Update liczby produktów po zapisie: `n_items++` .
7. Po wczytaniu całego pliku, tablica zawierać powinna `n_items` produktów. Sprawdzić poprawność zapisu, wyświetlając na ekranie każdy produkt z tablicy, używając poprzednio zdefiniowanych funkcji (na poprzednich zajęciach).

2. Poniższy kod (do przeklejenia) definiuje tablicę `char` rozmiaru `9 x 15` , zawierającą prosty "obrazek" złożony z symbolów `x` , i drukuje ją na ekranie:

```
char art[9][15] = {
    {'X','X','X','X','X','X','X','X','X','X','X','X','X','X','X'},
    {'X','X','X','X','X','X','X','X',' ','X','X','X','X','X','X'},
    {'X','X','X','X','X','X','X','X',' ','X','X','X','X','X','X'},
    {'X','X','X','X','X',' ',' ',' ',' ','X','X','X','X','X','X'},
    {'X','X','X','X','X','X','X',' ',' ','X','X','X','X','X','X'},
    {'X','X',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ','X','X','X'},
    {'X','X','X','X','X','X','X',' ',' ','X','X','X','X','X','X'},
    {'X','X','X','X','X','X','X',' ',' ','X','X','X','X','X','X'},
    {'X','X','X','X','X','X','X','X','X','X','X','X','X','X','X'}
};
// Print the art
for(int i = 0; i < 9; i++) {
    for(int j = 0; j < 15; j++) {
        printf("%c",art[i][j]);
    }
    printf("\n");
}
```

- o w swoim programie, pobrać od użytkownika dane o decyzji (0 lub 1), czy dokonać odbicia lustrzanego obrazka w poziomie (np. do zmiennej `int do_h_swap`) i w pionie (`int do_v_swap`).
- o jeśli `do_h_swap==1`, odwrócić obrazek w poziomie.
- o jeśli `do_v_swap==1`, odwrócić go też potem w pionie.
- o wyświetlić obrazek po transformacjach.

3. Zadeklarować tablicę dynamiczną `images` do przechowywania obrazków po każdej z transformacji jako wskaźnik na tablicę `char` rozmiaru `9 x 15`, to jest: `char (*images)[9][15]` (ten zapis na początku może być dość tajemniczy - jak ktoś chce zrozumieć, dlaczego to akurat działa, jak chcemy, przeczytanie, co to jest [right-left rule]([C Right-Left Rule \(Rick Ord's CSE 131 - UC San Diego\)](#)) w C - WYSOCE REKOMENDOWANE).

- o zaalokować za pomocą `malloc` odpowiednią liczbę bajtów, aby przechować pierwszy obraz (przed transformacjami odbić lustrzanych): `1*img_size`, gdzie:
 - `img_size = szer * wys * sizeof(char)`
 - w naszym przypadku `szer=15, wys=9`.
- o po każdej z transformacji, jakich zażyczył sobie użytkownik, powiększyć rozmiar tablicy `images` za pomocą `realloc`: jeśli dotychczas w tablicy mamy `img_len` obrazów, po rozszerzeniu powinna móc przechować `(img_len+1)*img_size` bajtów (liczba bajtów przedtem + jeden nowy obraz, patrz wzór w poprzedniej kropce). Obecnej długości tablicy `img_len`, czy też liczby obrazów do niej zapisanej, trzeba pilnować w kodzie samemu.
 - **UWAGA:** pamiętaj, aby zapisać wynik `realloc` z powrotem do zmiennej `images`.
- o ...po czym dopisać na koniec tablicy `images` (na nowo powstałe miejsce) dane obrazu zaraz po zastosowaniu transformacji.
- o wyświetlić na koniec wszystkie wersje obrazu: przed transformacjami i po zastosowaniu każdej z transformacji po kolei.

Uproszczony pseudokod:

```
rozmiar_obrazu := szer*wys*sizeof(char)
zaalokuj miejsce na jeden obraz w images
img_len := 1

przepisz art do images[0]:
    images[0][i][j] := art[i][j], dla każdego i,j-tego piksela

jeśli do_h_swap:
    realloc images na (img_len+1)*rozmiar_obrazu
    img_len++
    wykonaj poziome odbicie na art
    przepisz art do images[img_len-1]
jeśli do_v_swap:
    realloc images na (img_len+1)*rozmiar_obrazu
    img_len++
    wykonaj pionome odbicie na art
    przepisz art do images[img_len-1]
Wypisz na ekranie obrazki po podmianach:
for k in 0 to img_len-1:
    wypisz images[k]
```